

Métodos de Búsqueda para Sokoban

Análisis Experimental y Estadístico

Integrantes: Britez, Santiago; Kantor, Marcos; Sánchez Toledo, Mariano.
Sistemas de Inteligencia Artificial - ITBA

Instituto Tecnológico de Buenos Aires

August 25, 2026



Índice

Marco Teórico Fundamental

Representación del Estado

Algoritmos de Búsqueda

Heurísticas

Diseño Experimental

Resultados

Heurísticas

Variabilidad

Limitaciones

Conclusiones

Definición de Agente (Russell & Norvig)

"Un agente es todo aquello que puede considerarse que percibe y responde o actúa en un ambiente, con un cierto grado de independencia o autonomía."

- ▶ El agente busca maximizar su **función de desempeño**.
- ▶ En nuestro caso: el jugador de Sokoban es el agente, el tablero es el ambiente.
- ▶ Ambiente: **discreto, determinístico, estático, totalmente observable**.

Problema bien formado (Sección 4.3)

1. **Estados:** Configuraciones del tablero (posición jugador + cajas).
2. **Estado inicial:** s_0 (tablero dado).
3. **Acciones:** $\mathcal{A} = \{U, D, L, R\}$ (movimientos del jugador).
4. **Modelo de Transición:** $T(s, a) = s'$ (efecto de mover/empujar).
5. **Función de costo:** $g(s, a) = 1$ (cada movimiento cuesta 1).
6. **Condición de solución:** Todas las cajas en objetivos.

Observación

Sokoban es **NP-Hard** y **PSPACE-complete**, lo que justifica el uso de búsqueda informada (Sección 4.5.5).

Clase SokobanState

- ▶ `player`: tupla (fila, columna).
 - ▶ `boxes`: tupla de posiciones de cajas **ordenadas** (para hash único).
 - ▶ `walls`: conjunto de posiciones de paredes.
 - ▶ `goals`: conjunto de posiciones de objetivos.
 - ▶ `rows`, `cols`: dimensiones del tablero.
-
- ▶ `__hash__`: $h = \text{hash}(\text{player}, \text{boxes}) \rightarrow$ permite **Conjuntos de explorados**.
 - ▶ `__eq__`: Define unicidad del estado por $(\text{player}, \text{boxes})$.
 - ▶ Justificación: paredes y objetivos son **estáticos**, no varían.

Descripción del Problema (I/II)

Ambiente

- ▶ **Discreto:** Cantidad limitada de movimientos.
- ▶ **Ambiente Conocido:** Siempre se conoce la consecuencia de sus acciones (empuja la caja y se mueve a la dirección deseada).
- ▶ **Individual:** Solo 1 agente.
- ▶ **Totalmente Observable:** Toda la información está disponible.
- ▶ **Determinístico:** El estado siguiente del ambiente se define por el estado actual y la acción del agente.
- ▶ **Secuencial:** Se altera la posición de la caja y eso afecta las decisiones futuras.
- ▶ **Estático:** El ambiente no cambia mientras el agente no actúa.

Agentes

- ▶ **Racional:** Busca maximizar su medida de desempeño.
- ▶ **Con estado interno:** Todos mantienen estructuras de memoria (frontera, explorados) para decidir su próxima acción.

Clasificación de Métodos de Búsqueda

Algoritmo	Completo	Óptimo	Frontera
BFS	Sí	Sí (costo uniforme)	FIFO
DFS	No (sin límite)	No	LIFO
IDDFS	Sí	Sí	LIFO con límite
Greedy	No	No	Heap por $h(n)$
A*	Sí (con heur. admisible)	Sí (con heur. admisible)	Heap por $f(n)$

Teorema de A* (Sección 4.5.2)

Si $h(n)$ es admisible, A* es **óptimo** y **completo** (con ramificación finita y costos $> \varepsilon$).

Definición Formal de Heurística

Heurística (Definición 4.5.1)

Función $h(s)$ que estima el costo de la ruta más barata desde el estado s hacia un estado meta:

- ▶ Si s es Goal $\rightarrow h(s) = 0$
- ▶ Si no es Goal y $g(s, a) > 0 \rightarrow h(s) > 0$
- ▶ Si no es Goal y $g(s, a) \geq 0 \rightarrow h(s) \geq 0$

Heurística Admisble (Proposición 4.5.1)

Una heurística es **admisble** si **nunca sobrestima** el costo real:

$$h(n) \leq h^*(n) \quad \forall n$$

Donde $h^*(n)$ es el costo real mínimo.

Heurística h Manhattan: Suma de Distancias Manhattan

Fórmula

$$h_{\text{Manhattan}}(s) = \sum_{\text{cada caja } b} \min_{\text{objetivo } g} (|x_b - x_g| + |y_b - y_g|)$$

Explicación

Esta heurística suma, para cada caja, la distancia en línea recta (sin paredes) hasta el objetivo más cercano. Es como si las cajas pudieran volar solas hasta su destino, sin importar el jugador ni las otras cajas.

- ▶ **Admisible:** Sí, porque el costo real de mover una caja hasta un objetivo es *al menos* su distancia Manhattan (cada empuje la mueve una casilla). Nunca sobrestima.
- ▶ **Consistente:** Sí, cumple la desigualdad del triángulo.
- ▶ **Dominancia:** Es la heurística más débil de todas (ninguna otra está dominada por ella).
- ▶ **Ventajas:** Muy simple y rapidísima de calcular.
- ▶ **Desventajas:** Muy poco informativa; ignora por completo la interacción entre cajas, las paredes y la posición del jugador.

Heurística h Manhattan deadlock: Manhattan + Distancia del Jugador + Deadlocks

Fórmula

$$h_{MD}(s) = \begin{cases} \infty & \text{si hay deadlock} \\ h_{\text{Manhattan}}(s) + \max(0, \min_{box} \text{Manhattan}(player, box) - 1) & \text{en caso contrario} \end{cases}$$

Explicación

Esta heurística combina lo mejor de la distancia del jugador y la poda por deadlocks. Es la más completa de las tres.

- ▶ **Admisible:** Sí, porque tanto la parte de distancia como la poda por deadlocks son admisibles.
- ▶ **Consistente:** Sí.
- ▶ **Dominancia:** Domina a la Manhattan. Es la heurística más fuerte (mayor valor) de las admisibles.
- ▶ **Ventajas:** Máxima reducción de nodos entre las admisibles, manteniendo la garantía de optimalidad. En los experimentos, A* con hMD expandió un 53% menos nodos que con la Manhattan.
- ▶ **Desventajas:** Es la más costosa de calcular, pero la reducción de nodos compensa con creces.

Heurística h no admisible: $2 \times$ Manhattan

Fórmula

$$h_{\text{NoAdm}}(s) = 2 \cdot h_{\text{Manhattan}}(s)$$

Explicación

Esta heurística es un truco: toma la Manhattan y la duplica. Al hacerlo, *sobrestima* el costo real en muchos casos (por ejemplo, si la Manhattan ya era exacta, el doble es mayor que el costo real). Por eso no es admisible.

- ▶ **Admisible: No.** Viola la condición de no sobrestimación.
- ▶ **Consistente: No** (porque no es admisible, tampoco es consistente).
- ▶ **Dominancia:** No aplica (no es admisible).
- ▶ **Ventajas:** Muy rápida y reduce muchísimo los nodos expandidos (como vimos en los resultados).
- ▶ **Desventajas:** **Pierde la garantía de optimalidad** de A^* . Puede encontrar soluciones subóptimas o incluso no encontrar la óptima.

Importante

Esta heurística se incluye para mostrar el efecto de usar una heurística no admisible: ganas en velocidad pero sacrificas calidad. En la práctica, solo se usa si la optimalidad no es crítica.

Dominancia y Consistencia de Heurísticas

Dominancia (Definición 4.5.2)

h_{MD} domina a $h_{Manhattan}$ si $h_{MD}(n) \geq h_{Manhattan}(n) \forall n$. A^* con h_{MD} expande menos nodos.

Consistencia (Monotonicidad)

h es consistente si $h(n) \leq c(n, a, n') + h(n') \forall n, a, n'$.

- ▶ $h_{Manhattan}$ y h_{MD} son **consistentes**.
- ▶ h_{NoAdm} (no admisible) **no es consistente**.

Relación de dominancia:

$$h_{MD} \geq h_{Manhattan} \quad \text{y} \quad h_{NoAdm} \text{ no es admisible}$$

909 búsquedas comparan calidad, tiempo y memoria

Protocolo	Qué medimos
▶ 101 mapas: 1 personalizado y 100 estándares. ▶ 9 configuraciones: BFS, DFS, IDDFS, Greedy y A*. ▶ Corte: 200 000 nodos expandidos por ejecución. ▶ IDDFS: profundidad máxima de 100 niveles.	1. Tasa de éxito. 2. Costo de la solución. 3. Nodos expandidos. 4. Frontera máxima. 5. Tiempo de ejecución.

Pregunta experimental

¿Qué método ofrece la mejor relación entre calidad de solución y esfuerzo computacional?

A*-MD: mismo costo que BFS con 75% menos nodos

Método	Éxito	Costo med.	Nodos med.	Tiempo med.
A*-MD	100%	19	495	0.0082 s
BFS	100%	19	1947	0.0123 s
Greedy-MD	100%	26	84	0.0014 s
DFS	100%	48	803	0.0046 s
IDDFS	44.5%	16*	200000	0.6632 s

Frente a BFS

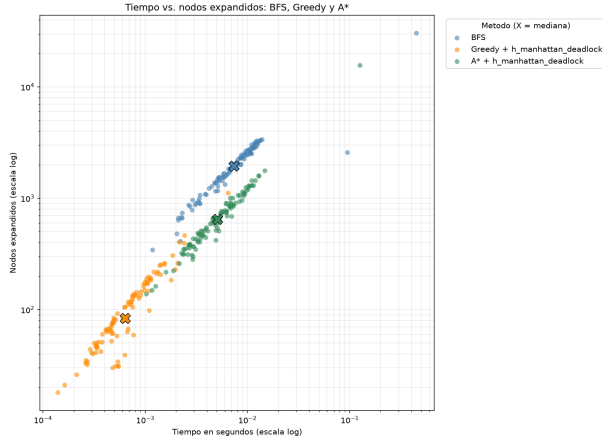
A*-MD obtiene el mismo costo mediano y reduce los nodos expandidos en **74.6%**.

Frente a Greedy

Greedy es casi **6 veces más rápido**, pero sus soluciones son **37% más largas**.

*El costo de IDDFS se calcula solo sobre los 45 mapas resueltos y no es directamente comparable.

Menos nodos suele implicar menos tiempo



Tres perfiles

- ▶ **Greedy-MD**: menor nodos y tiempo, soluciones más largas.
- ▶ **A*-MD**: mejor equilibrio.
- ▶ **BFS**: expande más nodos.

Resultado central

A*-MD conserva el tiempo de BFS con **74.6% menos nodos**.

Deadlocks reduce a la mitad el trabajo de A^*

Heurística	Nodos med.	Reducción	Tiempo med.	Admisible
Manhattan	1058	—	0.0147 s	Sí
Manhattan + deadlocks	495	53.2%	0.0082 s	Sí
No admisible	521	50.8%	0.0067 s	No

Por qué mejora

La detección de *deadlocks* identifica estados sin solución y evita expandir ramas inútiles.

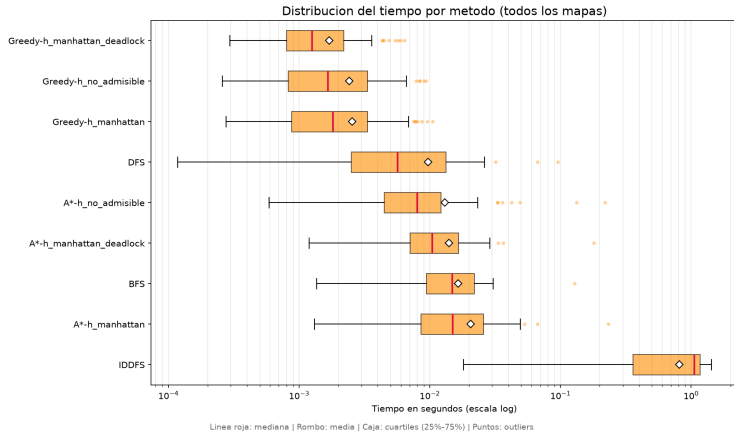
Por qué elegimos A^* -MD

Logra prácticamente la velocidad de la heurística no admisible, pero conserva la garantía de optimalidad.

Conclusión

El resultado empírico de una heurística no admisible puede ser óptimo, pero no existe garantía teórica de que siempre lo sea.

La distribución revela variabilidad y valores atípicos



Cómo leer el gráfico

- **Caja:** 50% central (P25–P75).
- **Línea roja:** mediana.
- **Rombo:** media.
- **Puntos:** valores atípicos.

Interpretación

Greedy concentra los menores tiempos. DFS presenta mayor dispersión y varios casos excepcionalmente lentos, mientras que IDDFS opera en una escala de tiempo claramente superior.

Expansiones y frontera miden aspectos diferentes

Método	Nodos med.	Frontera P95	Lectura
Greedy-MD	84	132	Poco trabajo y frontera pequeña
A*-MD	495	420	Menos expansiones que BFS
BFS	1947	396	Más expansiones; frontera similar
DFS	803	95	Frontera pequeña
IDDFS	200000	25	Frontera mínima; reexpande nodos

Nodos expandidos

Es el **trabajo acumulado**: cuántos estados fueron procesados durante toda la búsqueda.

Frontera máxima

Es el **pico de nodos pendientes**: cuántos estados esperaron simultáneamente.

Lectura correcta

A*-MD expande 74.6% menos nodos que BFS, aunque ambos alcanzan picos de frontera similares.

Los nodos expandidos son medianas; la frontera corresponde al percentil 95 de la frontera máxima por mapa.

Una frontera pequeña no garantiza un mejor método

Método	Frontera P95	Éxito	Costo med.	Resultado
Greedy-MD	132	100%	26	Rápido, aproximado
A*-MD	420	100%	19	Mejor equilibrio
BFS	396	100%	19	Óptimo, más trabajo
DFS	95	100%	48	Frontera pequeña, peor costo
IDDFS	25	44.5%	16*	Frontera pequeña, baja cobertura

Complejidad espacial teórica

BFS, A* y Greedy: $O(b^d)$.

DFS: $O(bm)$.

IDDFS: $O(bd)$.

Qué se midió realmente

La frontera es un **indicador parcial** de memoria. No se midió el consumo total de RAM.

Conclusión

A*-MD sigue ofreciendo el mejor equilibrio general. BFS tiene una frontera un poco mas pequeña pero mayor cantidad de nodos expandidos.

*Costo calculado únicamente sobre los 45 mapas resueltos por IDDFS.

IDDFS queda limitado por el protocolo

Resultado observado

- ▶ Resuelve **45 de 101 mapas**.
- ▶ Alcanza la mediana de **200 000 nodos**.
- ▶ Registra **0.66 s** de tiempo mediano.

Qué lo explica

- ▶ Reexpande nodos al aumentar el límite de profundidad.
- ▶ No utiliza información heurística para orientar la búsqueda.
- ▶ El protocolo impone profundidad 100 y 200 000 nodos.

Alcance de la conclusión

Bajo esta configuración, IDDFS no resulta práctico para Sokoban. La tasa de éxito no demuestra una falla intrínseca del algoritmo: también refleja los límites experimentales.

A*-MD ofrece el mejor equilibrio

Necesidad	Método	Trade-off
Solución óptima y eficiente	A*-MD	Recomendación general
Respuesta inmediata	Greedy-MD	Soluciones 37% más largas
Referencia óptima simple	BFS	Más nodos expandidos
Memoria muy limitada	DFS	Soluciones muy subóptimas

Tres conclusiones

1. A*-MD iguala el costo de BFS y reduce las expansiones en **74.6%**.
2. La detección de *deadlocks* reduce en **53.2%** los nodos de A* frente a Manhattan.
3. Greedy es la opción rápida cuando se acepta perder calidad de solución.

Ver solución completa (78 pasos)

¡Gracias por su atención!

"La heurística correcta no solo acelera la búsqueda: permite conservar la calidad de la solución mientras se evita explorar estados innecesarios."

